DIGITAL ELECTRONICS

Question 1. Simplify F = A'B + AB + AB'.

Step 1: Combine the first two terms: A'B + AB = B(A' + A).

Step 2: Since A' + A = 1, this becomes B.

Step 3: F = B + AB'. Using X + X'Y = X + Y, B + AB' = A + B.

Final answer: F = A + B.

Question 2. Simplify F = AB + A'C + BC using the consensus theorem.

The consensus theorem states XY + X'Z + YZ = XY + X'Z.

Here, X = A, Y = B and Z = C. Therefore AB + A'C + BC = AB + A'C; the term BC is redundant.

Final answer: F = AB + A'C.

Question 3. For F(A,B,C) = A + BC, write the truth table and canonical SOP.

Step 1: F is 1 whenever A = 1, so rows 100, 101, 110 and 111 produce 1.

Step 2: When A = 0, F = BC, so only row 011 produces 1.

Step 3: With ABC as the binary minterm index, the 1-rows are 3, 4, 5, 6 and 7.

Truth table: 000→0, 001→0, 010→0, 011→1, 100→1, 101→1, 110→1, 111→1.

Final answer: F = Σm(3,4,5,6,7).

Question 4. Convert F(A,B,C) = A'B + BC' into minterm notation.

Step 1: A'B is 1 when A=0 and B=1, for either value of C: 010 (m2) and 011 (m3).

Step 2: BC' is 1 when B=1 and C=0, for either value of A: 010 (m2) and 110 (m6).

Step 3: Take the union of the minterms and do not repeat m2.

Final answer: F = Σm(2,3,6).

Question 5. Minimize Σm(1,2,3,5,7) using a 3-variable K-map.

Step 1: Place 1s in cells m1, m2, m3, m5 and m7 using Gray-code order 00, 01, 11, 10.

Step 2: Group m1, m3, m5 and m7 as a group of four. In these cells C=1, so this group gives C.

Step 3: The remaining m2 can be paired with m3. A=0 and B=1 stay fixed, giving A'B.

Final answer: F = C + A'B.

Question 6. Minimize Σm(0,2,4,6,7) using a 3-variable K-map.

Step 1: Put 1s in m0, m2, m4, m6 and m7.

Step 2: Group m0, m2, m4 and m6 as a group of four. In all these cells C=0, giving C'.

Step 3: Pair m6 with m7. A=1 and B=1 stay fixed, giving AB.

Final answer: `F = C' + AB`. (The uploaded answer key's expression is incorrect.)

Question 7. Minimize Σm(0,1,2,3,8,9,10,11) using a 4-variable K-map.

Step 1: Mark all eight minterms listed in the K-map.

Step 2: These eight cells all have B=0; A, C and D vary freely.

Step 3: A group of eight eliminates A, C and D, leaving B'.

Final answer: `F = B'`.

Question 8. Minimize Σm(1,3,5,7,9,11,13,15) using a 4-variable K-map.

Step 1: The listed minterms are exactly the eight combinations whose least significant bit D=1.

Step 2: Make one group of eight in the K-map. A, B and C vary, while D stays 1.

Final answer: `F = D`.

Question 9. Minimize Σm(0,2,3,6,7,8,10,11,14,15) and identify essential prime implicants.

Step 1: The eight minterms 2,3,6,7,10,11,14,15 all have C=1. Group them as a group of eight to obtain C.

Step 2: The remaining minterms are 0 and 8. Pair each with adjacent 2 and 10 respectively; the four cells 0,2,8,10 form a group of four.

Step 3: In that group B=0 and D=0, giving B'D'.

Step 4: Both groups are needed to cover all required minterms. The prime implicants are C and B'D', and each covers required minterms not covered by the other.

Final answer: `F = C + B'D'`. Essential prime implicants: `C` and `B'D'`.

Question 10. Minimize Σm(4,5,6,7,12,13,14,15) using a 4-variable K-map.

Step 1: The eight minterms listed have B=1.

Step 2: Group all eight cells where B=1. A, C and D vary and disappear.

Final answer: `F = B`.

Question 11. Minimize Σm(1,3,7,9,11,15) + d(0,2,5,8,10).

Step 1: Place 1s at 1,3,7,9,11,15 and X (don't-care) marks at 0,2,5,8,10.

Step 2: Use the don't-cares to form a group of eight covering all cells with B=0. This group gives B'.

Step 3: The remaining required 1 at m15 is already covered by B' only if B=0; since m15 has B=1, group m3, m7, m11 and m15 as a group of four, giving CD.

Final answer: `F = B' + CD`. The don't-care cells are used only when they help simplify the function.

Question 12. Minimize Σm(0,2,8,10,11,14,15) + d(1,3,9,13).

Step 1: Mark the required 1s and the don't-care cells in the 4-variable K-map.

Step 2: The required minterms 0,2,8,10 and the available don't-cares 1,3,9,13 allow a large grouping for B=0, giving B'.

Step 3: The remaining required minterms 11,14 and 15 can be covered by grouping the C=1, A=1 cells where needed; a minimal expression is B' + AC.

Final answer: `F = B' + AC`. (Equivalent minimum forms may be possible depending on the selected don't-care groupings.)

Question 13. Explain minterm, maxterm, don't-care, prime implicant and essential prime implicant.

Minterm: An AND/product term containing every variable once, either complemented or uncomplemented. It is 1 for exactly one input combination.

Maxterm: An OR/sum term containing every variable once. It is 0 for exactly one input combination.

Don't-care: An input combination for which the output is not specified as either 0 or 1; it may be used as 0 or 1 to simplify a circuit.

Prime implicant: A largest possible K-map group that cannot be combined with another adjacent group to make a larger valid group.

Essential prime implicant: A prime implicant that covers at least one required 1-cell not covered by any other prime implicant.

Question 14. For Σm(2,3,6,7,10,11,14,15), find the largest groups and minimum expression.

Step 1: All listed minterms have C=1.

Step 2: Group all eight cells where C=1; A, B and D vary.

Final answer: `F = C`.

Question 15. 4-bit decimal input: output 1 for digits 0,1,2,3,8,9; inputs 10–15 are don't-cares.

Step 1: Use A as the most significant bit and D as the least significant bit. Required 1s are m0,m1,m2,m3,m8,m9; m10–m15 are X.

Step 2: Group m0,m1,m2,m3 with m8,m9 and suitable don't-cares to form the largest possible groups.

Step 3: A valid minimum expression is B'. It is 1 for the required digits 0,1,2,3,8,9; the other B'=1 combinations are among the don't-care inputs or must be checked against the specified truth table.

Final answer: `F = B'`.

Question 16. Design a Half Adder.

Step 1: A half adder adds two 1-bit inputs A and B and produces Sum S and Carry C.

Step 2: Truth table: AB=00→S0,C0; 01→S1,C0; 10→S1,C0; 11→S0,C1.

Step 3: Sum is 1 when inputs differ, so S = A ⊕ B. Carry is 1 only when both inputs are 1, so C = AB.

Step 4: Implementation: one XOR gate for Sum and one AND gate for Carry.

Final equations: `S = A ⊕ B; C = AB`.

Question 17. Design a Full Adder using two Half Adders and one OR gate.

Step 1: Half Adder 1 adds A and B: S1=A⊕B and C1=AB.

Step 2: Half Adder 2 adds S1 and Cin: Sum=S1⊕Cin=(A⊕B)⊕Cin and C2=S1Cin.

Step 3: OR the two carry outputs: Cout=C1+C2=AB+Cin(A⊕B).

Step 4: Expanding the carry expression gives Cout=AB+ACin+BCin.

Final equations: `Sum = A ⊕ B ⊕ Cin`; `Cout = AB + ACin + BCin`.

Question 18. Design a 4-bit Ripple Carry Adder.

Step 1: Connect four full adders in series, one for each bit (bit 0 through bit 3).

Step 2: Each stage receives Ai, Bi and the carry from the previous stage. The first stage receives the initial carry Cin.

Step 3: Each stage generates Si=Ai⊕Bi⊕Ci and Ci+1=AiBi+AiCi+BiCi.

Step 4: The carry from bit 0 must reach bit 1, then bit 2, then bit 3. This serial movement causes ripple carry propagation delay.

Design: Four cascaded full adders; outputs are S3S2S1S0 and final carry Cout.

Question 19. Design a Half Subtractor.

Step 1: A half subtractor computes A – B and produces Difference D and Borrow Bo.

Step 2: Truth table: AB=00→D0,Bo0; 01→D1,Bo1; 10→D1,Bo0; 11→D0,Bo0.

Step 3: Difference is XOR: D=A⊕B. Borrow occurs only when A=0 and B=1, so Bo=A'B.

Final equations: `D = A ⊕ B; Bo = A'B`.

Question 20. Design a Full Subtractor.

Step 1: A full subtractor computes A – B – Bin.

Step 2: Difference is `D = A ⊕ B ⊕ Bin`.

Step 3: Borrow-out is 1 when A is too small to subtract B and Bin. One standard form is `Bout = A'B + A'Bin + BBin`.

Step 4: It can be implemented using XOR gates for Difference and AND/OR logic for Borrow-out.

Final equations: `D = A ⊕ B ⊕ Bin`; `Bout = A'B + A'Bin + BBin`.

Question 21. Design a 4-to-1 Multiplexer using basic gates.

Step 1: A 4:1 MUX has data inputs I0–I3, select inputs S1,S0 and output Y.

Step 2: S1S0=00 selects I0; 01 selects I1; 10 selects I2; 11 selects I3.

Step 3: Gate each data input with the select combination that activates it, then OR the four terms.

Output equation: `Y=S1'S0'I0 + S1'S0I1 + S1S0'I2 + S1S0I3`.

Question 22. Implement F(A,B,C)=Σm(1,2,5,7) using a 4-to-1 MUX.

Step 1: Choose A and B as the select lines: S1=A and S0=B.

Step 2: Evaluate F for each AB combination. For AB=00, minterms 0 and 1 give F=C, so I0=C.

Step 3: For AB=01, minterms 2 and 3 give F=C', so I1=C'.

Step 4: For AB=10, minterms 4 and 5 give F=C, so I2=C.

Step 5: For AB=11, minterms 6 and 7 give F=C, so I3=C.

Final data inputs: `I0=C, I1=C', I2=C, I3=C`.

Question 23. Design a 3-to-8 Decoder with Enable E.

Step 1: Inputs are A,B,C and enable E. When E=0, every output is 0.

Step 2: When E=1, exactly one output is 1 for each input combination.

Step 3: The output equations are:

`Y0=EA'B'C'; Y1=EA'B'C; Y2=EA'BC'; Y3=EA'BC;`

`Y4=EAB'C'; Y5=EAB'C; Y6=EABC'; Y7=EABC.`

Step 4: Implement each output with an AND gate using the required true/complemented input signals and E.

Question 24. Design a 2-to-4 Decoder using NAND gates; explain active-low outputs.

Step 1: A and B form four combinations: 00, 01, 10 and 11.

Step 2: Generate A' and B' with input inverters, or NAND-based inverters.

Step 3: For a NAND implementation with active-low outputs, use NAND gates on the decoded input combinations. The output corresponding to a selected input combination goes LOW, while the others remain HIGH.

Step 4: Active-low means the asserted/selected output is logic 0 rather than logic 1. Exact gate arrangement depends on whether complemented inputs and output polarity are available.

Question 25. Design a 4-to-2 Encoder.

Step 1: The four inputs are I0,I1,I2,I3; the two outputs encode the active input's index in binary.

Step 2: Assuming exactly one input is HIGH, I0→00, I1→01, I2→10 and I3→11.

Step 3: Output Y1 is 1 for I2 or I3, so Y1=I2+I3. Output Y0 is 1 for I1 or I3, so Y0=I1+I3.

Step 4: If more than one input is HIGH, the basic encoder may produce an ambiguous code. A priority encoder is needed if simultaneous inputs must be handled.

Final equations: `Y1=I2+I3; Y0=I1+I3`.

Question 26. Design a 2-bit Magnitude Comparator.

Step 1: Let A=A1A0 and B=B1B0. Compare the most significant bits first.

Step 2: Equality requires both bit pairs to match: `A=B=(A1 XNOR B1)(A0 XNOR B0)`.

Step 3: A>B if A1=1,B1=0, or if the MSBs are equal and A0=1,B0=0.

`A>B = A1B1' + (A1 XNOR B1)A0B0'`

Question 26. Design a 2-bit Magnitude Comparator.

Step 1: Let A=A1A0 and B=B1B0. Compare the most significant bits first.

Step 2: Equality requires both bit pairs to match: `AeqB=(A1 XNOR B1)(A0 XNOR B0)`.

Step 3: A is greater than B if A1=1,B1=0, or if the MSBs are equal and A0=1,B0=0.

`AGTB = A1B1' + (A1 XNOR B1)A0B0'`

Step 4: A is less than B if A1=0,B1=1, or if the MSBs are equal and A0=0,B0=1.

`ALTB = A1'B1 + (A1 XNOR B1)A0'B0`.

Question 27. Design a 4-bit Even Parity Generator.

Step 1: Even parity requires the total number of 1s including the parity bit to be even.

Step 2: XOR of the four data bits is 1 when the data contains an odd number of 1s.

Step 3: Choose the parity bit equal to that XOR result so the total count becomes even.

Final equation: P = A XOR B XOR C XOR D.

Question 28. Design a 5-bit Even Parity Checker.

Step 1: The checker receives four data bits A,B,C,D and the parity bit P.

Step 2: XOR all five bits: E=A XOR B XOR C XOR D XOR P.

Step 3: For valid even parity, the XOR result is 0. If E=1, the received word has odd parity and an error is detected.

Final equation: E=A XOR B XOR C XOR D XOR P; E=0 means valid even parity.

Question 29. Output 1 when at least three of A,B,C,D are 1.

Step 1: The output is 1 for every combination containing exactly three 1s or four 1s.

Step 2: The combinations with exactly three 1s are 1110, 1101, 1011 and 0111; 1111 is also included.

Step 3: A minimized sum-of-products expression is:

Final expression: F = ABC + ABD + ACD + BCD.

Step 4: Each term represents a set of three inputs that are 1. If any three or all four inputs are 1, at least one term is true.

Question 30. Design a 4-bit ALU supporting ADD, AND, OR and XOR.

Step 1: Use two 4-bit inputs A[3:0] and B[3:0], plus two operation-select bits S1,S0.

Step 2: Build four parallel function blocks: a 4-bit adder, four AND gates, four OR gates and four XOR gates.

Step 3: Connect the four 4-bit results to a 4-to-1 multiplexer for each output bit.

Step 4: One valid operation-select assignment is 00=ADD, 01=AND, 10=OR, 11=XOR. The mapping may be changed if the design specification requires it.

Step 5: The MUX selects the result sent to the ALU output. The adder may also provide a carry-out flag.

Block architecture: A,B → {Adder, AND, OR, XOR} → 4:1 result MUX → F[3:0]; S1,S0 control the MUX.